Similarity Is Not Authorisation: Need-to-Know Collective Memory for Multi-Team LLM Agents

[Student Name]*
Department of Computer Science and Engineering, [Institute Name]
[student-email]

[Guide Name]
Department of Computer Science and Engineering, [Institute Name]
[guide-email]

Abstract

Organisations now run fleets of LLM agents, one per team, each with a private memory. Pooling those memories promises that no team re-derives what another already knows—but a shared memory is searched by *similarity*, and similarity is not authorisation. We formalise the requirement as *retrieval non-interference* (RNI): everything a principal observes from a memory search—which items, in what order, with what scores, and how many—must be a function of the memories it may read. We prove that the prevailing *rank-then-filter* design violates RNI through the *number* of results it returns, and that hybrid retrieval with index-wide lexical statistics violates it through *scores* even when every returned item is authorised; *authorise-before-rank* with document-local scoring satisfies it. We build SML, a governed collective memory for agents on this principle: one access predicate compiled into application, SQL and vector-index form, document-local hybrid scoring, a defence-in-depth re-check, immutable versions, contradiction review and an append-only audit trail, reachable over REST, SDKs and the Model Context Protocol. Controlled experiments confirm the analysis: when ten teams hold topically overlapping knowledge and half of it is private, rank-then-filter answers 99.1 % of queries with a truncated list—each revealing that similar hidden memories exist—and it still does so for 32.4 % of queries when teams' knowledge is topically disjoint; index-wide BM25 changed the scores of visible results in 100.0 % and their order in 36.9 % of cases when only hidden memories changed; and filtered HNSW matched exact authorise-before-rank on every query at no measurable cost. End to end, 0 of 140 adversarial attempts leaked, sharing raised cross-team Hit@1 from 0.35 to 0.975, and on LoCoMo, LongMemEval and BEIR SML outperformed BM25 and a plain vector store (Recall@10 0.673 and 0.807; nDCG@10 0.718 and 0.359), matching the benchmark authors' DRAGON retriever with a 33M-parameter local model at 83–109 ms median latency. Answers that an open 8B model wrote from SML's memories were judged correct for 72.6 % of 135 sampled LoCoMo questions, against 67.4 % with BM25, 54.8 % with a plain vector store ($p < 0.001$) and 0.0 % without memory.

1 Introduction

AI agents now write code, test releases, triage security findings and answer customers. They are typically configured per team, and each keeps its own memory: a conversation log, a scratch file, a private vector index. When the Development agent learns that sign-ins fail because refresh tokens expire, the QA agent that meets the same symptom a week later cannot benefit from

*Replace the bracketed placeholders with the author details before submission.

it; it re-investigates, pays for the same reasoning again, and may record a different conclusion. In our in-house study, agents restricted to their own memory had to re-derive 62.5 % of the answers the organisation already held (Section 6.4).

The obvious remedy is a single shared memory. It is also where the difficulty begins. Memory retrieval ranks *everything in the index* by similarity to the query, and a Support agent asking why PDF export fetches internal URLs is, by construction, closest to the Security team's confidential server-side request forgery finding. *Similarity is not authorisation.* The common engineering answer—retrieve the top candidates, then drop the ones the caller may not read—looks safe, because no forbidden item is ever returned. We show that it is not: the *number* of results it returns depends on memories the caller cannot see, which turns every search into an oracle for the existence of hidden knowledge, and it silently loses the caller's own relevant memories when hidden ones crowd the candidate list. Hybrid retrieval adds a second, subtler channel: lexical scores such as BM25 use collection-wide statistics, so the *scores and order* of perfectly authorised results move when hidden memories change—the same class of leak Büttcher and Clarke demonstrated for multi-user file-system search two decades ago [1].

This paper makes the requirement precise and builds a system that meets it.

1. **A security property for memory retrieval.** We define *retrieval non-interference* (RNI), an adaptation of non-interference [2] to ranked retrieval, and prove that authorise-before-rank with document-local scoring satisfies it, rank-then-filter violates it through result counts, and index-wide scoring statistics violate it through scores (Section 3).
2. **SML, a governed collective memory for agents.** One access predicate over five visibility levels is compiled into application code, a SQL predicate and a vector-index filter; hybrid dense and full-text retrieval uses only document-local scores; every candidate is re-checked against the system of record; knowledge is versioned, contradictions are routed to human review, and every access and refusal is written to an append-only log. Agents connect over REST, SDKs and the Model Context Protocol [3] (Section 4).
3. **Controlled experiments on the placement of authorisation**: recall starvation and short-list rates of rank-then-filter on two multi-tenant corpora, the statistics side channel of index-wide BM25, and the fidelity and cost of filtered approximate search (Sections 6.1–6.2).
4. **An end-to-end evaluation**: an adversarial suite of 140 attacks, end-to-end answer accuracy with an LLM judge, an in-house cross-team benchmark, and three public benchmarks (LoCoMo [4], LongMemEval [5], BEIR [6]) against BM25, a plain vector store and published results of other memory layers (Sections 6.3–6.7).

The implementation, the benchmark harnesses and all raw results are released with the project.

2 Background and Threat Model

Agent memory today. Research systems give a single agent long-term memory by retrieval over an event stream [7] or by paging context in and out of the prompt [8]. Production memory layers—Mem0 [9], Zep [10], LangMem [11], A-Mem [12]—extract facts or temporal knowledge graphs with an LLM and are evaluated by the answer accuracy they enable on long conversations. Their unit of isolation is a user, agent or project identifier: memory is either wholly private to that scope or wholly visible within it. Organisation-wide sharing with per-memory need-to-know rules, and the leakage it can cause, is outside their design and their evaluations.

Principals, memories and policy. An organisation has teams. A *principal* p is an agent (authenticated by its own API key) or a human user; it belongs to at most one team $\tau(p)$ and holds roles that grant permissions such as `memory:create`, `memory:read` or `memory:share`. A *memory* m has content, a type, tags, importance and confidence in $[0, 1]$, an owning team

$\tau(m)$, a creator $c(m)$, a visibility $v(m)$, a sensitivity level, an optional expiry, explicit grants to teams $G_T(m)$ and agents $G_A(m)$, and a version history. Visibility is one of `private`, `team`, `selected_teams`, `selected_agents` or `organization` (Appendix, Table 7); confidential and restricted memories may not be organisation-wide. For a non-administrator holding `memory:read`, an active, unexpired memory is readable, $R(p, m)$, if and only if

$$\begin{aligned} & c(m) = p \ \vee \ v(m) = \texttt{organization} \\ & \vee \ \Big(v(m) \neq \texttt{private} \ \wedge \ \big[(\tau(p) = \tau(m) \wedge v(m) \in \{\texttt{team}, \texttt{selected_teams}\}) \\ & \qquad \vee \ \tau(p) \in G_T(m) \ \vee \ p \in G_A(m) \big] \Big). \end{aligned} \tag{1}$$

Modifying a memory additionally requires its creator or a manager of the owning team.

Threat model. The adversary is an authenticated but unauthorised principal: an agent of another team, or a compromised agent, that issues arbitrary API requests. It may search with any text, including the exact title or content of a memory it wants (the strongest similarity attack), probe the system repeatedly, attempt to update, version-browse or share memories to itself, and attempt privilege escalation (creating teams or agents, reading the audit log, listing users, re-indexing). It observes everything the API returns: result lists, their lengths, scores, errors and warnings. Timing side channels, compromise of the database or host, and poisoning of shared knowledge by authorised writers [13] are out of scope; the latter is mitigated but not prevented by provenance, verification and contradiction review.

3 Retrieval Non-Interference

3.1 Definitions

Let S be the set of stored memories and $\mathcal{V}_p(S) = \{m \in S : R(p, m)\}$ the memories principal p may read. A retrieval mechanism F maps a principal, a query q and the store to an observable response $F(p, q, S)$: an ordered list of memories with scores (and hence a length). A scoring function $s(q, m)$ is *document-local* if its value depends only on q and on attributes of m itself, and not on any other memory in S. Cosine similarity of embeddings, PostgreSQL's `ts_rank_cd`, importance, confidence and verification status are document-local; BM25 [14] is not, because its inverse document frequency and length normalisation are computed over a collection.

Definition 1 (Confidentiality of content)**.** *F is* content-confidential *if $F(p, q, S) \subseteq \mathcal{V}_p(S)$ for every p, q and S.*

Definition 2 (Retrieval non-interference)**.** *F satisfies* retrieval non-interference *(RNI) if for every principal p, query q and pair of stores S, S' with $\mathcal{V}_p(S) = \mathcal{V}_p(S')$, the responses are identical: $F(p, q, S) = F(p, q, S')$, including order, scores and length.*

RNI is the retrieval analogue of Goguen–Meseguer non-interference [2, 15]: changing what p cannot see must not change what p observes. Content confidentiality is necessary but, as we show, far from sufficient.

Figure 1 shows three placements of authorisation relative to ranking, with top_k selecting the k highest-scoring memories (ties broken by a fixed memory order):

$$\begin{aligned} F_{\mathrm{none}}(p, q, S) &= \mathrm{top}_k(s, S), \\ F^{n}_{\mathrm{post}}(p, q, S) &= \mathrm{top}_k\big(s, \ \mathrm{top}_n(s, S) \cap \mathcal{V}_p(S)\big) \qquad (n \geq k), \\ F_{\mathrm{pre}}(p, q, S) &= \mathrm{top}_k\big(s, \ \mathcal{V}_p(S)\big). \end{aligned}$$

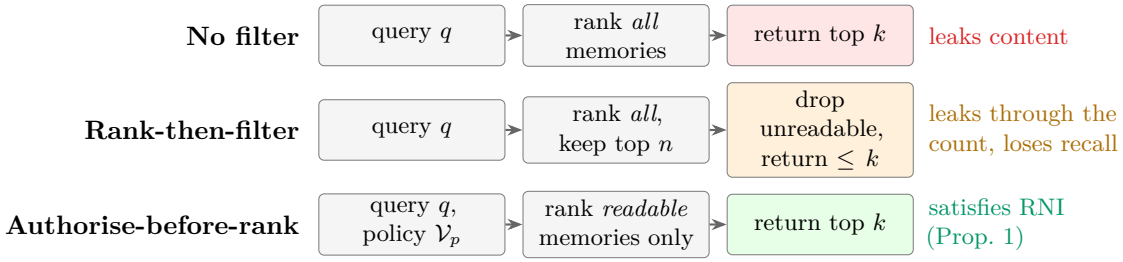


Figure 1: Three placements of authorisation in memory retrieval.

3.2 Results

Proposition 1 (Authorise-before-rank)**.** *If s is document-local, F_{pre} is content-confidential and satisfies RNI.*

Proof. F_{pre} selects from $\mathcal{V}_p(S)$ only, so it is content-confidential. Its output is determined by the set $\mathcal{V}_p(S)$, the scores $s(q, m)$ for $m \in \mathcal{V}_p(S)$ and the fixed tie-breaking order. Because s is document-local, each score depends on q and m alone. Hence if $\mathcal{V}_p(S) = \mathcal{V}_p(S')$ the candidate sets and all their scores coincide, and so do the selected lists, their order, scores and length. □

Proposition 2 (Rank-then-filter)**.** *F^n_{post} is content-confidential but, for every finite depth $n \geq k$, violates RNI: there exist stores S, S' with $\mathcal{V}_p(S) = \mathcal{V}_p(S')$ whose responses differ in length.*

Proof. Content confidentiality holds because the output is a subset of $\mathcal{V}_p(S)$. Let S contain at least k memories, all readable by p, so that $F^n_{\text{post}}(p, q, S)$ has length k. Let $S' = S \cup H$, where H consists of $n - k + 1$ memories hidden from p that score higher on q than every memory of S (for instance paraphrases of q written by another team). Then $\mathcal{V}_p(S') = \mathcal{V}_p(S)$, but $\text{top}_n(s, S')$ consists of H and only $k - 1$ readable memories, so $F^n_{\text{post}}(p, q, S')$ has length $k - 1$. □

Corollary 1 (Existence oracle and recall starvation)**.** *Under rank-then-filter, an unauthorised principal learns whether hidden memories similar to a query of its choice exist by observing a short result list; and readable memories that would rank in the top k among $\mathcal{V}_p(S)$ are dropped whenever hidden memories occupy the first n ranks. Over-fetching (larger n) raises the number of similar hidden memories needed to trigger either effect, but no finite n removes them.*

Proposition 3 (Collection statistics)**.** *If s depends on statistics of S that include hidden memories—inverse document frequency, average length, or normalisation by the maximum score over S—then even F_{pre} can violate RNI: responses are content-confidential but their scores and order depend on hidden memories.*

Proof. For BM25, the weight of term t is $\log(1 + (N - \text{df}_t + 0.5)/(\text{df}_t + 0.5))$ with N and df_t counted over S. Adding hidden memories containing t changes df_t and N, hence the score of every readable memory containing t, although $\mathcal{V}_p(S)$ is unchanged; with several query terms the relative weights shift and the order of readable memories can change. □

Büttcher and Clarke showed that such statistics are exploitable: in multi-user file search, relevance scores computed over a system-wide index let a user infer the content of files it cannot read [1]. Proposition 3 says the same channel re-appears in hybrid agent memory whenever the lexical component is BM25 over the shared index—the default in many hybrid search engines.

Remark (Approximate search). Graph indexes such as HNSW [16] traverse hidden vectors while answering a filtered query; filtered traversal [17, 18] returns only matching vectors, so content confidentiality holds, but the approximation error can in principle depend on the hidden part of the graph. Proposition 1 is exact for exhaustive search; we measure how closely filtered HNSW matches it in Section 6.1.

Remark (Declassification). A system may deliberately release a bounded amount of information about hidden memories. SML does so at exactly one point: when a new memory contradicts one its author cannot read, the author learns only the *count* of such conflicts, the details go to administrators, and the event is audited.

4 SML: Design

SML is built so that every path to a memory realises F_{pre} with document-local scores.

4.1 Architecture

Figure 2 shows the components. Every request passes one gateway that authenticates it (a JWT [19] or an agent API key) and authorises it twice: role-based access control [20] decides whether the operation is permitted, and the read predicate (1)—an attribute-based rule [21]—decides whether the specific memory may be touched. PostgreSQL is the system of record for memories, versions, grants, identities and the audit log; Qdrant [22] stores one vector per memory with the payload fields the predicate needs; embeddings are computed locally with `bge-small-en-v1.5` [23] (384 dimensions, 33M parameters), so no text leaves the deployment.

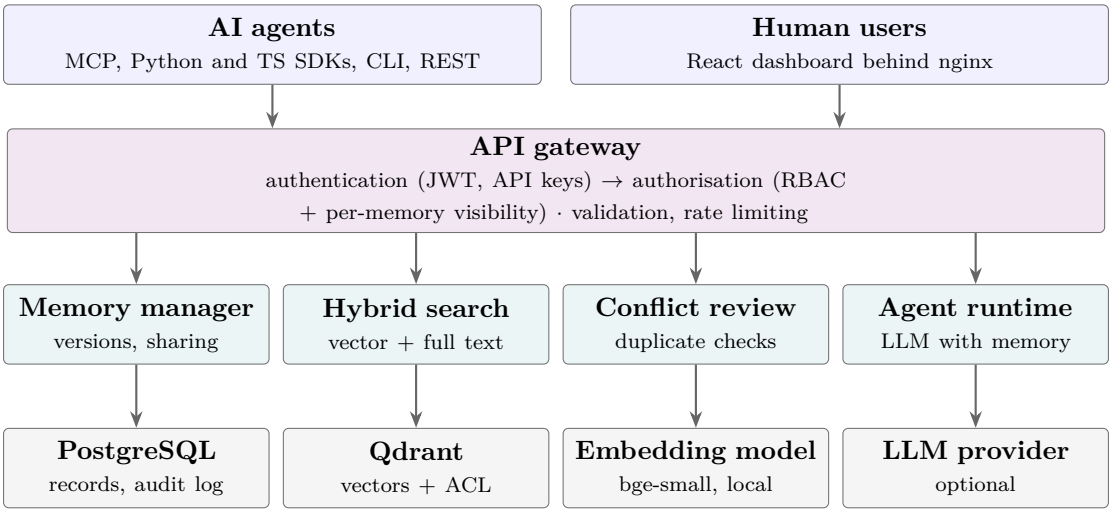


Figure 2: Architecture of SML. Every path to a memory passes authentication and two-level authorisation; audit records are written in an independent transaction.

4.2 One predicate, three enforcement points

The read predicate (1) exists in three synchronised forms, and tests assert that they agree: `can_read` evaluates a concrete PostgreSQL row and is authoritative; `sql_read_filter` is the same rule as a SQL predicate for listings and full-text search; `build_read_filter` expresses it as a Qdrant payload filter—a disjunction of conjunctions over visibility, owning team, creator, grants, status and expiry—that is pushed *into* the nearest-neighbour query. Similarity search therefore ranks only readable memories (F_{pre}). Every candidate is then re-checked with `can_read` against the current row, which protects against a stale vector payload after a permission change and against a bug in either filter.

4.3 Document-local hybrid retrieval

Embeddings capture paraphrase but blur exact identifiers such as error codes, CVE numbers and ticket ids. SML adds a PostgreSQL full-text path [24] (Algorithm 1): a generated, weighted

`tsvector` over title and content, queried with `websearch_to_tsquery` under the SQL form of the same predicate, in which every query term must match. Both paths use only document-local signals, as Proposition 3 requires:

$$s(m) = 0.90\,\mathrm{sim}(q,m) + 0.05\,\mathrm{imp}(m) + 0.05\,\mathrm{conf}(m) + 0.02\,[\mathrm{verified}(m)] + 0.10\,\mathrm{kw}(q,m), \quad (2)$$

where sim is cosine similarity (computed from Qdrant for keyword-only candidates too) and kw is `ts_rank_cd`—a cover-density rank that uses no collection statistics—normalised by the best keyword score among the *readable* candidates. The weights favour meaning, let verified and high-confidence knowledge win ties, and reward exact matches. If the vector store is unreachable, search degrades to permission-filtered keyword search instead of failing.

Algorithm 1 Permission-aware hybrid search (F_{pre} with document-local scores)

Require: principal p, query q, metadata filters F, result size k
$\phi \leftarrow \textsc{ReadFilter}(p) \wedge F$ ▷ visibility, grants, status, expiry, type, project, ...
$C_{\mathrm{kw}} \leftarrow$ full-text matches of q in PostgreSQL under ϕ (all terms), ranked by `ts_rank_cd`
$\mathbf{v} \leftarrow \textsc{Embed}(q)$
$C_{\mathrm{vec}} \leftarrow$ the $3k$ nearest neighbours of $\mathbf{v}$ in Qdrant under ϕ
$C \leftarrow C_{\mathrm{kw}} \cup C_{\mathrm{vec}}$; obtain cosine similarity for keyword-only candidates
for all $m \in C$ **do**
  if not $\textsc{CanRead}(p, m)$ on the current PostgreSQL row **then**
    remove m from C ▷ defence in depth
  end if
  compute $s(m)$ with Equation (2)
end for
update access and cross-team reuse counters; append a `MEMORY_SEARCH` audit record
return the k highest-scoring memories of C

4.4 Governance of shared knowledge

Writes. A new memory is validated, authorised (`memory:create`; the body may not name another agent or team), embedded and checked against *readable* memories for duplicates (identical content hash, or cosine similarity ≥ 0.92). Organisation-wide candidates with similarity ≥ 0.70 are passed to contradiction rules (e.g. different numeric values or opposing technology choices for the same subject), which open a conflict for human review; this is the declassification point of Section 3. The row, version 1 and grants are flushed to PostgreSQL, the vector is upserted, and the transaction commits; if the commit fails the vector is deleted, and an administrator endpoint detects and repairs any divergence by re-embedding from PostgreSQL.

Versions and provenance. Updates lock the row, check an optional expected version, and store an immutable snapshot linked to its predecessor; there is one vector per memory, so search always returns the latest version. Humans resolve conflicts with *keep new*, *keep existing*, *keep both* or *dismiss*; nothing is overwritten silently.

Accountability. Logins, token events, every memory operation, sharing and role changes, conflict resolutions and every denial are recorded with actor, team, on-behalf-of principal, resource, outcome, reason and request identifier. Audit records are written in an independent transaction, so refused operations are always recorded, and a database trigger rejects `UPDATE` and `DELETE` on the audit table.

Identity. Passwords are hashed with Argon2id [25]; access tokens live 15 minutes; refresh tokens rotate on every use and re-use revokes the whole token family; agent API keys are stored only as keyed hashes; authentication is rate limited with temporary lock-out.

4.5 Agent interfaces

Agents reach SML through a REST API (57 operations), a Model Context Protocol endpoint exposing eight tools (`search_memory`, `remember`, `get_memory`, `update_memory`, `share_memory`, `memory_history`, `list_teams`, `whoami`) that off-the-shelf MCP clients can use without code, and Python and TypeScript SDKs with a command-line tool, bulk import and LLM tool definitions. All interfaces act as the calling agent through the same services, so authorisation and auditing are identical on every path.

5 Implementation

SML is written in Python 3.12 with FastAPI (API), SQLAlchemy 2 and Alembic (PostgreSQL 16 schema with foreign keys, check constraints, a generated full-text column and the append-only audit trigger), Qdrant 1.13 with payload indexes on every filtered field, and fastembed/ONNX for local embeddings. A React dashboard provides monitoring, a memory explorer, access management, the audit log, conflict review and an *agent playground* in which a human acts as an agent under that agent's permissions, audited on the human's behalf. The system runs as four containers from one `docker compose` command. Its test suite—105 backend tests against real PostgreSQL, Qdrant and the embedding model (including an end-to-end MCP test with the official client), 14 Python and 5 TypeScript SDK tests, and a 16-step acceptance scenario against the running deployment—passes.

6 Evaluation

We ask seven questions. **RQ1:** where should authorisation happen—what do rank-then-filter and filtered approximate search cost? **RQ2:** do index-wide ranking statistics leak? **RQ3:** is SML secure end to end? **RQ4:** does shared memory help agents? **RQ5:** how good is SML's retrieval on public benchmarks, and how does it compare with other memory layers? **RQ6:** does better retrieval yield more correct answers? **RQ7:** what does governed retrieval cost?

Setup. All experiments ran on one laptop (Windows 11 on ARM64, Docker Desktop with 12 virtual CPUs and 15.5 GB of memory, no GPU). RQ1–RQ2 are controlled experiments with the system's embedding model and exact cosine search (and Qdrant for filtered HNSW). RQ3–RQ7 exercise the running system through its public API; the public benchmarks ran against a second API process with its own database and collection, so 100,000+ benchmark memories did not touch the demonstration data. Two baselines ran over exactly the same texts: **BM25** (k_1=1.5, b=0.75) and a **plain vector store** using `all-MiniLM-L6-v2` [26, 27], the default embedding model of Chroma-style memories. Published results of other systems are quoted, not re-run.

Validity checks. Our local `all-MiniLM-L6-v2` baseline reproduces its published BEIR scores exactly (nDCG@10 0.645 on SciFact and 0.316 on NFCorpus [28]); SML's dense mode reproduces the published scores of `bge-small-en-v1.5` (0.717 vs. 0.713; 0.342 vs. 0.343); our LongMemEval metrics, ported line by line from the official code, agree with it to 10^{-16} on 3,000 randomised rankings; and retrieval is deterministic—two independent LoCoMo runs produced identical recall.

6.1 RQ1: Where should authorisation happen?

Design. We turn two corpora into multi-tenant memories with ten teams. In *LoCoMo-MT*, each of the 10 LoCoMo conversations (5,882 turns, 1,535 questions) belongs to its own team and every question is asked by the team that owns its conversation: teams hold *topically disjoint* knowledge. In *SciFact-MT*, the 5,183 abstracts and 300 claims of BEIR SciFact are assigned to teams uniformly at random: teams hold *topically overlapping* knowledge, as departments of one organisation do. A fraction ρ of every other team's memories is private to that team; the rest is organisation-visible. We compare F_{pre}, F^{n}_{post} with over-fetch $n = \alpha k$ for $\alpha \in \{1, 2, 5, 10\}$, and F_{none}, all with $k = 10$ and exact cosine search, over three random seeds. Recall@10 is measured against the evidence the asker may read.

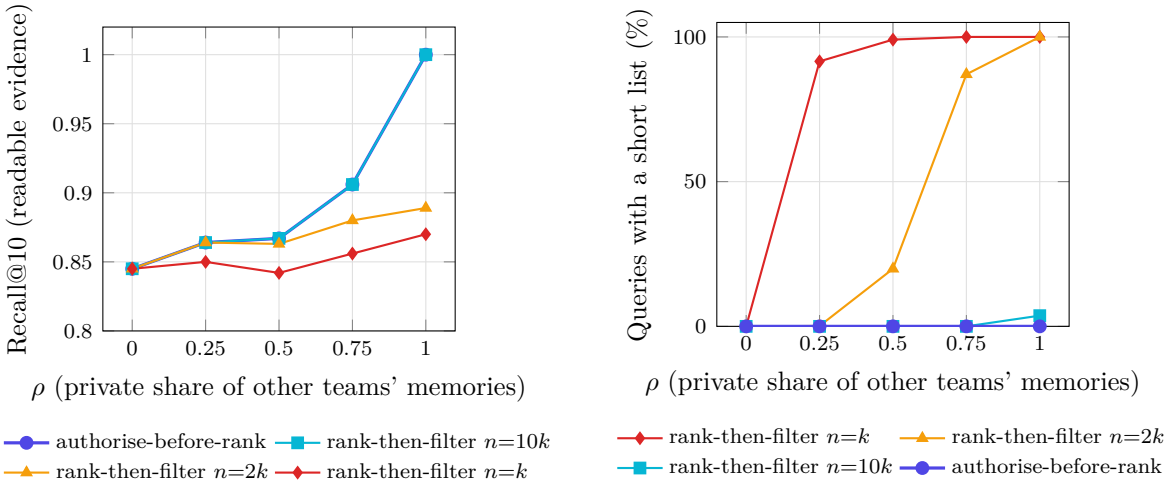


Figure 3: SciFact-MT (ten teams, topically overlapping knowledge). Left: recall of the evidence the asker may read. Right: share of queries answered with fewer than $k = 10$ results—the observable behind Proposition 2. On the left, the $n = 10k$ curve coincides with authorise-before-rank. Questions without readable evidence are excluded, so the sample shrinks from 900 question–seed pairs at $\rho = 0$ to 538 at $\rho = 0.5$ and 81 at $\rho = 1$.

Table 1: Placement of authorisation (exact cosine, $k = 10$, mean over three seeds). n: question–seed pairs with readable evidence. Recall@10 is over readable evidence; *short*: share of queries answered with fewer than 10 results; *leaked*: share of F_{none} results the asker may not read.

			Recall@10				Short lists		Leaked
Corpus	ρ	n	F_{pre}	F^{k}_{post}	F^{2k}_{post}	F^{10k}_{post}	F^{k}_{post}	F^{10k}_{post}	F_{none}
LoCoMo-MT	0.5	4,605	**0.569**	0.562	0.568	0.569	24.8 %	0.0 %	5.1 %
LoCoMo-MT	1.0	4,605	**0.584**	0.562	0.576	0.583	32.4 %	0.5 %	10.2 %
SciFact-MT	0.5	538	**0.867**	0.842	0.863	0.867	99.1 %	0.0 %	42.2 %
SciFact-MT	1.0	81	**1.000**	0.870	0.889	1.000	100.0 %	3.7 %	83.3 %

Findings. Figure 3 and Table 1 confirm Proposition 2. The short-list channel is pervasive: with overlapping knowledge and half of it private (SciFact-MT, $\rho = 0.5$), rank-then-filter with $n = k$ answers 99.1 % of queries with fewer than ten results (on average 5.8), and each short list tells the asker that hidden memories similar to its query exist. Even with topically disjoint knowledge (LoCoMo-MT, $\rho = 1$) 32.4 % of answers are short. Over-fetching helps but is not a fix: with $n = 10k$ the channel disappears at $\rho = 0.5$, yet when all other teams' knowledge is private 3.7 % of SciFact-MT queries still come back short—and those return no result at all. Recall suffers as well: readable evidence that authorise-before-rank finds is crowded out of the

candidate list, costing 0.867 → 0.842 at $\rho = 0.5$ and 1.000 → 0.870 at $\rho = 1$ on SciFact-MT, and 0.584 → 0.562 on LoCoMo-MT; over-fetching five to ten times recovers it in these corpora. Without any filter, 42.2 % of the results on SciFact-MT at $\rho = 0.5$, and 10.2 % on LoCoMo-MT at $\rho = 1$, are other teams' private memories. Authorise-before-rank always returns k results with the best recall and nothing to observe.

Filtered approximate search. We loaded LoCoMo-MT into Qdrant with an HNSW index covering every vector and issued 500 filtered queries at two selectivities (Table 2). Filtered HNSW never returned an unreadable memory, and its top-10 lists agreed with exact F_{pre} in 100.0 % ($\rho = 0.5$) and 100.0 % ($\rho = 1$, where the asker may read only 10.2 % of the index) of positions, with median latencies of 1.5 and 1.2 ms against 1.4 and 1.3 ms for the same queries without a filter. Authorising inside the index is therefore exact in practice and free.

Table 2: Filtered HNSW in Qdrant on LoCoMo-MT (5,882 vectors, 500 queries, $k = 10$; client-side latency).

ρ	Readable share	Unreadable results	Agreement with exact F_{pre}	p50 filtered	p50 unfiltered
0.5	55.3 %	0	100.0 %	1.5 ms	1.4 ms
1.0	10.2 %	0	100.0 %	1.2 ms	1.3 ms

6.2 RQ2: Do index-wide ranking statistics leak?

We ranked the memories readable by the asker in LoCoMo-MT ($\rho = 0.5$) with BM25 whose document frequencies, collection size and average length are computed over the *whole* index, as a shared lexical index would, and then removed all hidden memories of one other team—a change the asker must not be able to observe. For 1,200 question–seed pairs, the scores of the visible top-10 results changed in 100.0 % of cases, the *order* of the visible top 10 changed in 36.9 %, and even the *set* of visible top-10 results changed in 11.7 %. Under the same perturbation, SML's scores are unchanged by construction: cosine similarity, `ts_rank_cd`, importance, confidence and verification status are document-local, and keyword scores are normalised over readable candidates only (Proposition 3). A shared lexical index must either compute statistics per principal, as Büttcher and Clarke proposed for file search [1], or avoid collection statistics altogether.

6.3 RQ3: Is SML secure end to end?

Every agent outside the authorised teams attacked each of seven protected memories through six paths, and four agents each attempted five privilege escalations (Table 3). No attempt leaked, including the 40 searches that used the protected memory's own title or content as the query; those memories were never candidates. All 100 expected denials were found in the audit log (the 40 search attempts are ordinary, audited searches rather than denials).

6.4 RQ4: Does shared memory help agents?

Four freshly created team agents (Development, QA, Security, Support) stored 35 labelled memories; 40 paraphrased questions have ground-truth answers, and 25 of them can only be answered with another team's knowledge. *Isolated* restricts each agent to its own memories, *Shared* is SML, and *Lexical* ranks exactly the memories the agent may read with TF-IDF, separating the effect of sharing from that of embeddings (Table 4).

Sharing makes cross-team questions answerable at all (Hit@5 0.375 → 1.000), and embeddings improve *ranking* over keyword matching on the same authorised memories (Hit@1 0.850

Table 3: Adversarial suite against the running system.

Attempt	Attempts	Blocked
Direct read of a protected memory	20	20
Update	20	20
Read version history	20	20
Share the memory to one's own team	20	20
Semantic search using the memory's title	20	20
Semantic search using the memory's full content	20	20
Privilege escalation (create team / agent, read audit log, list users, re-index)	20	20
Total	**140**	**140**

Table 4: In-house cross-team benchmark (35 memories, 40 queries, top 5).

Metric	Isolated	**Shared (SML)**	Lexical (TF-IDF)
Hit@1	0.350	**0.975**	0.850
Hit@5	0.375	**1.000**	**1.000**
MRR	0.358	**0.983**	0.921
nDCG@5	0.338	**0.957**	0.927
Hit@5 on the 25 cross-team queries	0.000	**1.000**	**1.000**
Answers reusing another team's knowledge	0.000	**0.675**	–
Tasks resolved from existing memory	0.375	**1.000**	–
Tasks whose knowledge had to be re-derived	0.625	**0.000**	–

$\rightarrow$ 0.975). Isolated agents re-derived 62.5 % of their answers; SML's duplicate detector flagged all 25 re-stored memories. Without an LLM, shared tasks took 122 ms against 265 ms including re-derivation; with a small local LLM (`qwen2.5:0.5b` on CPU) the shared agent was about 20 % slower (4,283 vs. 3,574 ms) because it reads more context, but every answer was grounded in existing organisational knowledge.

6.5 RQ5: Retrieval quality on public benchmarks

Authorisation is only worth having if retrieval stays good. We ran three public benchmarks end to end through the API (Figure 4; full tables in the appendix).

LoCoMo [4] has very long multi-session conversations with annotated evidence turns; it is the benchmark on which Mem0, Zep, LangMem and A-Mem report. Over the 1,535 answerable questions of the 10 released conversations, SML's hybrid search reaches Recall@10 0.673, within 0.01 of the DRAGON retriever [29] reported by the benchmark authors at every depth (Figure 5), although DRAGON has about three times as many parameters and was evaluated on the full 50-conversation set. Its margin over BM25 is largest on multi-hop and open-domain questions (+0.21 Recall@10), which paraphrase their evidence.

LongMemEval [5] gives each question its own chat history of about 48 sessions. Following the official protocol on LongMemEval$_S$ (419 questions, 102,229 stored user messages; Recall@k requires *all* evidence messages in the top k), SML reaches Recall@10 0.807 against 0.742 for BM25 and 0.711 for the plain vector store, and places all evidence within the top 50 for 97.4 % of questions. BM25 is marginally ahead on temporal and user-fact questions, which repeat exact names and dates—the price of SML's conservative, document-local keyword component (Section 7).

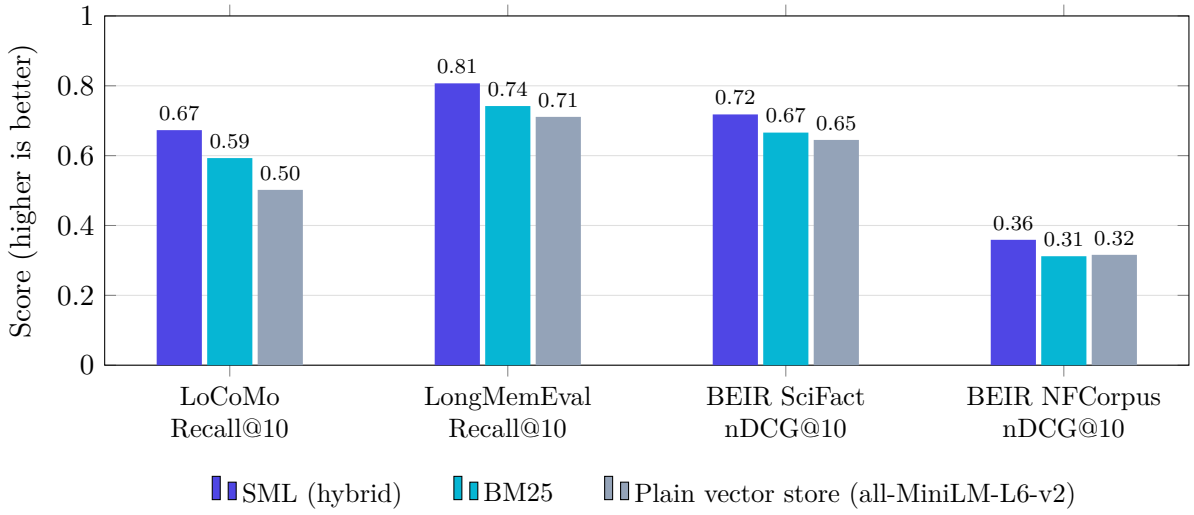


Figure 4: Headline metric of each public benchmark, measured through SML's API, and two baselines on identical data.

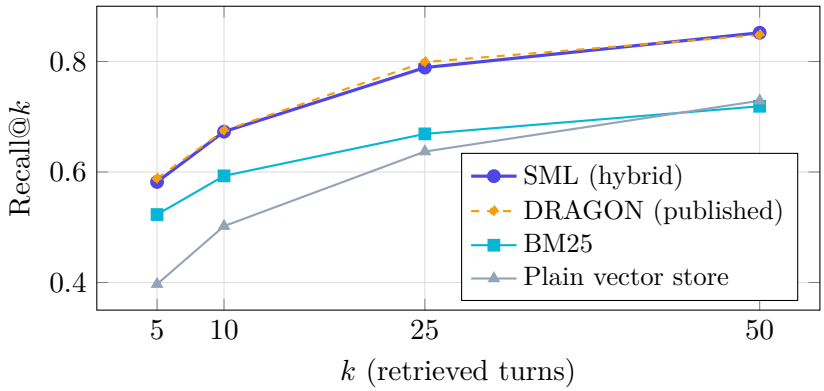


Figure 5: LoCoMo Recall@k through SML's permission-checked API.

BEIR [6] measures zero-shot document retrieval. SML scores nDCG@10 0.718 on SciFact and 0.359 on NFCorpus—above BM25, ColBERT [30], a BM25 plus cross-encoder pipeline and Chroma's default embedder, and within 0.016 and 0.024 of OpenAI's hosted `text-embedding-3-small`, Mem0's default embedder.

Other memory layers. Mem0, Zep, LangMem and A-Mem report LoCoMo *answer accuracy* judged by an LLM (Table 5) with the hosted GPT-4o-mini over all questions; SML's answer accuracy (RQ6) was measured with a local open model on a sample, so it is comparable with the configurations we ran, not with these numbers. On LongMemEval$_S$ Zep reports 71.2 % answer accuracy with GPT-4o (60.2 % for GPT-4o reading the full history) [10]. None of these evaluations reports access-control leakage or auditability.

Table 5: LoCoMo results of other memory layers as published in the Mem0 paper [9] (GPT-4o-mini as answerer and judge, all questions), and SML as measured. Published latencies include network access to hosted services. ‡Gemma 4 8B (local) as answerer and judge on 135 sampled questions (RQ6): comparable with the configurations we ran, not with the rows above.

System	Answer accuracy (LLM judge)	Search p50 / p95	Context tokens	Leakage tested
Full context (no memory layer)	72.9 %	–	26,031	–
Mem0 with graph memory	68.4 %	476 / 657 ms	3,616	no
Mem0	66.9 %	148 / 200 ms	1,764	no
Zep	66.0 %	513 / 778 ms	3,911	no
LangMem	58.1 %	17,990 / 59,820 ms	127	no
A-Mem	48.4 %	668 / 1,485 ms	2,520	no
SML (hybrid)	72.6 %‡	**83 / 186 ms**	≈441	**yes, 0 / 140**

6.6 RQ6: Does better retrieval yield more correct answers?

Protocol. We follow the protocol of the Mem0 paper [9]: an LLM answers each LoCoMo question from the memories a system retrieves, and an LLM judge labels the answer CORRECT or WRONG against the gold answer, with the adversarial category excluded. We make one substitution: instead of the hosted GPT-4o-mini, an open model that runs on the laptop, Gemma 4 [31] (`gemma4:latest`: 8.0B parameters, 4-bit quantisation, served by Ollama at temperature 0), acts as both answerer and judge, as one model does in the Mem0 protocol. The answerer receives the top-10 memories of a configuration, each with its date, and is asked for a short answer; the judge prompt, like Mem0's, accepts different wording and date formats. Because CPU generation takes about 25 s per answer, we drew a stratified random sample of 200 of the 1,540 answerable questions (proportional to the four categories, fixed seed) in random order and evaluated the first 135 of them—73 single-hop, 29 temporal, 28 multi-hop and 5 open-domain; the remaining questions were not run within the available compute time, and because the order was random the evaluated questions are themselves a random sample. Every evaluated question is answered under four contexts—SML's hybrid search, BM25, the plain vector store, and no memory—so the configurations are compared on identical questions with the exact McNemar test [32]; intervals are 95 % Wilson intervals [33].

Findings. Answers grounded in SML's memories were judged correct for 72.6 % of the questions, against 67.4 % with BM25, 54.8 % with the plain vector store and 0.0 % without memory (Table 6); without retrieved context the model declined or guessed wrongly on every question,

Table 6: LoCoMo answer accuracy with an LLM judge (Gemma 4 8B as answerer and judge, 135 stratified questions, top-10 memories as context). *Evidence retrieved*: accuracy on the questions for which at least one annotated evidence turn was among the ten memories. McNemar tests are paired against SML.

Context given to the answerer	Accuracy J	95 % CI	Tokens	J when evidence retrieved	McNemar vs SML
SML hybrid search	**72.6 %**	64.5 %–79.4 %	513	81.9 %	–
BM25	67.4 %	59.1 %–74.7 %	441	80.8 %	$p = 0.296$
Plain vector store	54.8 %	46.4 %–63.0 %	445	81.2 %	$p < 0.001$
No memory	0.0 %	0.0 %–2.8 %	0	–	$p < 0.001$

so all answer quality here comes from memory. SML is significantly more accurate than the plain vector store: 30 questions were answered correctly only with SML's context and 6 only with the vector store's ($p < 0.001$). Its lead over BM25 (20 against 13 such questions) is not significant at this sample size ($p = 0.296$). The ordering follows retrieval: whenever at least one evidence turn was among the ten memories, the three retrievers led to almost the same accuracy (81.9 %, 80.8 % and 81.2 %), and SML put evidence into the context for 77.8 % of the questions against 73.3 % for BM25 and 59.3 % for the vector store. The differences in answer accuracy therefore come from what each retriever puts into the context, not from how the answerer uses it.

Is the judge reliable? We audited 60 randomly drawn judgments by hand, comparing each generated answer with the gold answer under the judge's own rubric. The judge agreed with the audit on 86.7 % of them (Cohen's $\kappa = 0.708$ [34]); of the disagreements, 6 credited a related but different answer and 2 rejected an equivalent one. LLM judges are known to be lenient [35], so the absolute accuracies are probably a few points high; because the judge never sees which context produced an answer, there is no reason for this bias to differ between contexts. The audit labels are released with the results. Since answerer, judge and sample differ from the Mem0 paper's, these accuracies rank the four contexts against each other; they are not comparable with the published GPT-4o-mini numbers in Table 5.

6.7 RQ7: What does governed retrieval cost?

Median client-side search latency stayed between 83 and 109 ms from 35 to 102,229 memories, with p95 below 210 ms (Appendix, Table 15); server-side time—query embedding, filtered vector search, full-text query, per-candidate re-check, ranking, access counters and the audit write—was 62 ms at the median on LoCoMo. Filtering inside the index did not measurably change vector-search latency (RQ1). Creating a memory with duplicate and contradiction detection took 189 ms at the median; bulk import without duplicate detection reached 14–23 memories per second on the laptop CPU, bounded by embedding.

7 Discussion

Lessons for memory and vector-database designers. (i) *Push the policy into the index.* Filtered approximate search matched exact authorise-before-rank at no measurable cost (RQ1), whereas rank-then-filter leaks through result counts and loses recall; over-fetching recovers recall in practice but only raises the number of similar hidden memories an attacker needs. (ii) *Keep scores document-local, or compute statistics per principal.* Hybrid engines that compute BM25 over the shared index leak through scores even when every returned item is authorised (RQ2).

(iii) *Re-check against the system of record.* Vector payloads are replicas; permissions change. (iv) *Name your declassification points* and audit them.

Limitations. SML's keyword component rewards only memories containing every query term, which keeps it document-local but conservative; a BM25-weighted lexical ranking with statistics restricted to the readable set, fused with the dense ranking [36], should close the gap on name- and date-heavy questions. Answer accuracy was measured with a local 8B model on 135 sampled questions rather than with GPT-4o-mini on all of them, so its absolute level is not comparable with published J scores, and the full-context baseline (about 21,000 tokens per question) was not run on the CPU. Published results of other systems were not reproduced and come from different hardware, hosted services and, for DRAGON and Stella, larger data. The in-house corpus is small and author-written, which is why the public benchmarks were added. Latencies come from one laptop with co-located services, and rate limiting was disabled on the benchmark process only. RNI as defined ignores timing; approximate indexes can, in principle, reveal hidden structure through recall differences, which we measured but did not bound. Finally, SML does not consolidate or summarise memories with an LLM; LongMemEval reports that LLM-extracted fact keys add about 0.09 Recall@10.

Broader impact. Shared agent memory concentrates organisational knowledge. Need-to-know retrieval, audit trails and human conflict review make that concentration governable; they do not prevent an authorised agent from misusing what it may read, nor protect against poisoned knowledge written by an authorised principal [13].

8 Related Work

Memory for LLM agents. Generative Agents [7] and MemGPT [8] established retrieval-based long-term memory for a single agent. Mem0 [9], Zep [10], LangMem [11] and A-Mem [12] build production memory layers around LLM-based extraction and evaluate answer accuracy on LoCoMo [4] and LongMemEval [5]. They scope memory by user, agent or project; SML targets organisation-wide sharing under per-memory need-to-know rules and evaluates leakage. Multi-agent frameworks such as AutoGen [37], MetaGPT [38] and CAMEL [39] share context within a run; SML acts as a persistent, governed blackboard [40] across runs and teams (Appendix, Table 8).

Privacy and security of retrieval-augmented generation. Retrieval databases leak through generated text [41] and membership inference [42], and can be poisoned to steer answers [13]. These works study a single-owner corpus exposed to users of one application; we study multi-principal corpora in which the retrieval step itself must respect per-item permissions, and we formalise what that step may reveal.

Secure search over shared indexes. Büttcher and Clarke showed that a system-wide full-text index with post-processing leaks file contents to users who lack read permission, and proposed a security model in which statistics are computed over what the user may see [1]. Our Propositions 2 and 3 carry this line of work to dense and hybrid retrieval for agents and measure both channels. Non-interference [2] and language-based information flow [15] provide the underlying notion.

Filtered vector search. Filtered-DiskANN [17] and ACORN [18] make approximate nearest-neighbour search efficient under label or arbitrary predicates, and vector databases such as Qdrant evaluate payload filters inside HNSW [16]. SML uses such filtering as a security mechanism and shows that it is both RNI-preserving in practice and nearly free.

Retrieval models and access control. Dense retrievers [27, 29, 43], embedding models [23, 26], BM25 [14], late interaction [30], retrieval-augmented generation [44] and the BEIR and MTEB benchmarks [6, 28] underlie our retrieval and evaluation; RBAC [20] and ABAC [21] underlie our policy model.

9 Conclusion

A shared memory is only as trustworthy as its most permissive retrieval path. We defined retrieval non-interference, showed that the familiar rank-then-filter design and index-wide lexical statistics both violate it, and built SML, a collective memory for multi-team agents in which authorisation happens before ranking, scores are document-local, every result is re-checked, and knowledge is versioned, reviewed and audited. Controlled experiments confirmed each proposition; end to end, no adversarial attempt leaked, sharing turned unanswerable cross-team questions into answered ones, retrieval matched the benchmark authors' own retriever with a small local model, and answers written from SML's memories were judged correct for 72.6 % of sampled questions—significantly more often than with a plain vector store (54.8 %) and ahead of BM25 (67.4 %). Need-to-know and good recall are not in tension: placed correctly, authorisation costs nothing.

Artifact. The system, the benchmark harnesses (in-house, LoCoMo, LongMemEval, BEIR, and the placement experiments) and all raw results are part of the project repository.

References

[1] Stefan Büttcher and Charles L. A. Clarke. A security model for full-text file system search in multi-user environments. In *Proceedings of the 4th USENIX Conference on File and Storage Technologies (FAST)*, 2005.

[2] Joseph A. Goguen and José Meseguer. Security policies and security models. In *IEEE Symposium on Security and Privacy*, pages 11–20, 1982.

[3] Anthropic. Model context protocol specification. `https://modelcontextprotocol.io`, 2024.

[4] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[5] Di Wu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025.

[6] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[7] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[8] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[9] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[10] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

[11] LangChain. LangMem: Long-term memory for LLM agents. `https://langchain-ai.github.io/langmem/`, 2025.

[12] Wujiang Xu et al. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

[13] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. In *Proceedings of the 34th USENIX Security Symposium*, pages 3827–3844, 2025.

[14] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[15] Andrei Sabelfeld and Andrew C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003.

[16] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[17] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, et al. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference (WWW)*, 2023.

[18] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data (SIGMOD)*, 2(3), 2024.

[19] Michael B. Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). RFC 7519, Internet Engineering Task Force, 2015.

[20] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996.

[21] Vincent C. Hu, David Ferraiolo, Rick Kuhn, et al. Guide to attribute based access control (ABAC) definition and considerations. Technical Report NIST SP 800-162, National Institute of Standards and Technology, 2014.

[22] Qdrant Team. Qdrant: Vector similarity search engine. `https://qdrant.tech/documentation/`, 2024.

[23] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.

[24] The PostgreSQL Global Development Group. PostgreSQL 16 documentation: Full text search. `https://www.postgresql.org/docs/16/textsearch.html`, 2023.

[25] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. Argon2: New generation of memory-hard functions for password hashing and other applications. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 292–302, 2016.

[26] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[27] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[28] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2023.

[29] Sheng-Chieh Lin, Akari Asai, Minghan Li, Barlas Oguz, Jimmy Lin, Yashar Mehdad, Wen-tau Yih, and Xilun Chen. How to train your DRAGON: Diverse augmentation towards generalizable dense retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[30] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020.

[31] Gemma Team, Google DeepMind. Gemma 4 model card. `https://ai.google.dev/gemma/docs/core/model_card_4`, 2026.

[32] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[33] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[34] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[35] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[36] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

[37] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[38] Sirui Hong et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

[39] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[40] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition, 2009.

[41] Shenglai Zeng, Jiankun Zhang, Pengfei He, et al. The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 4505–4524, 2024.

[42] Maya Anderson, Guy Amit, and Abigail Goldsteen. Is my data in your retrieval database? membership inference attacks against retrieval augmented generation. In *Proceedings of the 11th International Conference on Information Systems Security and Privacy (ICISSP)*, 2025.

[43] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, et al. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[44] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[45] Dun Zhang. Stella en 1.5B v5 text embedding model. `https://huggingface.co/dunzhang/stella_en_1.5B_v5`, 2024.

A Additional Tables

Table 7: Memory visibility levels.

Visibility	Readable by (besides the creator and administrators)
`private`	nobody else; private memories cannot be shared
`team`	members of the owning team, plus explicit team or agent grants
`selected_teams`	the owning team plus the explicitly allowed teams
`selected_agents`	only the explicitly allowed agents
`organization`	every authenticated principal

Table 8: Capabilities of related approaches (✓ supported, ∼ partial / session-scoped / via extensions, – not addressed), assessed from the designs described in the cited papers and documentation.

Approach	Persistent memory	Cross-team sharing	Semantic retrieval	Per-memory access control	Versions & provenance	Security audit trail
AutoGen, MetaGPT, CAMEL	∼	∼	∼	–	–	–
Generative Agents, MemGPT	✓	–	✓	–	–	–
RAG over a vector store	✓	∼	✓	∼	–	–
Mem0	✓	∼	✓	∼	∼	–
Zep	✓	∼	✓	–	∼	–
SML (this work)	✓	✓	✓	✓	✓	✓

Table 9: LoCoMo evidence retrieval through SML's API (1,535 questions). Bold: best measured value per column.

Configuration	R@5	R@10	R@25	R@50	Hit@1	MRR	nDCG@10
SML hybrid	**0.582**	**0.673**	**0.789**	**0.852**	**0.392**	**0.511**	**0.523**
SML dense only	0.561	0.657	0.783	0.848	0.352	0.478	0.494
SML keyword only (all terms)	0.119	0.121	0.122	0.123	0.118	0.126	0.118
BM25	0.523	0.593	0.669	0.719	0.345	0.450	0.463
Plain vector store	0.397	0.502	0.637	0.729	0.212	0.326	0.344
Recency (last k turns)	0.002	0.010	0.034	0.078	0.001	0.005	0.004
DRAGON (published, 50 conv.)	*0.588*	*0.675*	*0.799*	*0.848*	–	–	–

Table 10: LoCoMo Recall@10 by question category (number of questions).

Configuration	Single-hop (841)	Temporal (320)	Multi-hop (282)	Open-domain (92)
SML hybrid	**0.733**	**0.736**	**0.488**	**0.473**
SML dense only	0.711	0.720	0.485	**0.473**
BM25	0.699	0.688	0.277	0.267
Plain vector store	0.570	0.562	0.309	0.266

Table 11: LongMemEval$_S$ retrieval (419 questions, official protocol). Stella V5 1.5B [45] was evaluated by the LongMemEval authors on LongMemEval$_M$, whose haystack is about ten times larger; it is a reference point only.

	Message level					Session level
Configuration	R@5	NDCG@5	R@10	NDCG@10	R@50	R@10
SML hybrid	**0.637**	0.663	**0.807**	**0.708**	**0.974**	0.907
SML dense only	**0.637**	0.656	0.802	0.700	**0.974**	**0.909**
BM25	0.628	**0.664**	0.742	0.693	0.840	0.783
Plain vector store	0.511	0.570	0.711	0.623	0.936	0.840
Stella V5 1.5B on M (published)	*0.582*	*0.481*	*0.692*	*0.512*	–	*0.783*

Table 12: LongMemEval$_S$ message-level Recall@10 by question type (number of questions).

Configuration	Multi-session (121)	Temporal (127)	Knowl. update (72)	User facts (64)	Preferences (30)	Assistant (5)
SML hybrid	**0.711**	0.732	**0.958**	0.953	**0.800**	**1.000**
SML dense only	**0.711**	0.724	**0.958**	0.938	**0.800**	**1.000**
BM25	0.562	**0.748**	0.931	**0.969**	0.500	0.800
Plain vector store	0.545	0.614	0.903	0.953	0.767	**1.000**

Table 13: BEIR nDCG@10. Upper block measured through the SML API; lower block published on the same test sets (BEIR paper [6] and MTEB results [28]).

Retriever	Source	SciFact	NFCorpus
SML hybrid (bge-small-en-v1.5, local)	measured	**0.718**	**0.359**
SML dense only	measured	0.717	0.342
BM25 (this evaluation)	measured	0.666	0.312
Plain vector store, all-MiniLM-L6-v2 (this evaluation)	measured	0.645	0.316
OpenAI text-embedding-3-small (Mem0's default embedder)	MTEB	0.734	0.383
OpenAI text-embedding-ada-002	MTEB	0.727	0.370
bge-small-en-v1.5	MTEB	0.713	0.343
BM25 + cross-encoder re-ranker	BEIR	0.688	0.350
ColBERT	BEIR	0.671	0.305
BM25 (Elasticsearch)	BEIR	0.665	0.325
all-MiniLM-L6-v2 (Chroma's default embedder)	MTEB	0.645	0.316
TAS-B	BEIR	0.643	0.319
ANCE	BEIR	0.507	0.237
DPR	BEIR	0.318	0.189

Table 14: LoCoMo answer accuracy by question category (Gemma 4 8B answerer and judge; number of evaluated questions). The open-domain column has very few questions and is shown for completeness only.

Context	Single-hop (73)	Temporal (29)	Multi-hop (28)	Open-domain (5)
SML hybrid search	87.7 %	37.9 %	67.9 %	80.0 %
BM25	83.6 %	37.9 %	53.6 %	80.0 %
Plain vector store	61.6 %	37.9 %	53.6 %	60.0 %
No memory	0.0 %	0.0 %	0.0 %	0.0 %

Table 15: Client-side search latency of SML's hybrid mode through the API (milliseconds).

Workload	Memories in the collection	p50	p95
In-house benchmark (top 5)	35	103	208
LoCoMo (top 50)	5,882	83	186
BEIR NFCorpus (top 50)	3,633	90	180
BEIR SciFact (top 50, long abstracts)	5,183	109	197
LongMemEval$_S$ (top 50)	102,229	84	189